

Cloud Storage Misconfiguration Assessment

Testing Unauthorized Access via an Over-Permissive Backblaze B2 Application Key

RED TEAM

Name	Sarah A. Thompson
Index Number	249
Course Code	CY376 — Network Monitoring, Security and Auditing
Track	Red Team
Topic	Cloud Storage Misconfiguration Testing
GitHub Repository	https://github.com/St-April01/ testing-cloud
Date	August 3, 2026

Abstract

Cloud storage misconfigurations remain one of the most common root causes of real-world data breaches, and among these, over-provisioned access credentials are a persistent and under-tested risk. This project investigates, from a Red Team perspective, how a cloud storage Application Key issued with broader permissions than an application legitimately requires can be discovered through a public source-code leak and subsequently abused to read and modify private data. A private Backblaze B2 bucket was provisioned and an Application Key was deliberately generated with account-wide, read-write scope. The keyID and applicationKey were embedded in a realistic .env configuration file and pushed to a public GitHub repository, simulating the single most common vector by which developer credentials leak into the wild. Using only the leaked key pair, an attacker-simulated session was able to download a private file and inject a new file into the bucket without any prior authorization, using the AWS Command Line Interface against Backblaze's S3-compatible endpoint.

A remediation key, scoped to a single bucket rather than the entire account, was then generated and tested under the same conditions. The scoped key was denied when attempting to enumerate all buckets on the account (an AccessDenied / "not entitled" response), demonstrating that bucket-level least-privilege scoping was successfully enforced; however, the same key retained both read and write access within its own bucket, showing that scope restriction and permission-level restriction are two distinct controls that must both be configured deliberately. The assessment concludes with a set of practical recommendations — key scoping, minimum-necessary permissions, secret scanning, and credential rotation — that map directly to the observed gaps and that are broadly applicable to any S3-compatible cloud storage service, not just Backblaze B2.

Table of Contents

Abstract.....	2
1. Introduction.....	4
2. Literature and Tooling Review.....	5
3. Methodology.....	6
4. Implementation.....	8
5. Results and Findings — Unauthorized Access.....	13
6. Remediation and Validation.....	17
7. Analysis and Recommendations.....	20
8. Conclusion.....	21
References.....	23
Appendix A — Full Command Log (Leaked Key).....	24
Appendix B — Full Command Log (Scoped Key).....	24
Appendix C — .env File Used to Simulate the Leak.....	24

1. Introduction

1.1 Background

Cloud object storage services such as Amazon S3, Google Cloud Storage, and Backblaze B2 have become the default place organizations keep everything from customer records to internal backups. Their popularity comes from convenience: a bucket can be created and made reachable over HTTPS in minutes, and access is normally controlled by two independent layers — the bucket's own access policy, and the credentials (Application Keys, in Backblaze's terminology) issued to applications and developers that need to reach it.

Public breach reports consistently point to misconfiguration, rather than a flaw in the storage service itself, as the proximate cause of cloud data exposure. Two sub-categories dominate: buckets left publicly readable by mistake, and credentials that are more powerful than the application using them actually needs — often because it was faster, during development, to issue a broad key than to scope a narrow one. The second category is comparatively under-discussed in student cybersecurity coursework, which tends to focus on the more visually obvious "public bucket" scenario. This project deliberately targets the credential-scoping problem instead, because it maps directly onto a very common real-world leak vector: secrets accidentally committed to source control.

1.2 Project Objective

The objective of this assessment is to demonstrate, end-to-end and under controlled lab conditions, how an over-permissive Backblaze B2 Application Key can be exposed through a public GitHub repository and then used by an unauthorized party to read and write data inside a private bucket, and to evaluate whether replacing that key with a bucket-scoped credential meaningfully reduces the resulting risk.

Specifically, the project set out to:

1. Provision an isolated Backblaze B2 bucket and issue an Application Key with unnecessarily broad (account-wide, read-write) scope.
2. Simulate a realistic credential leak by committing the key pair, inside a .env file, to a public GitHub repository.
3. From an attacker-equivalent position — holding only the leaked key pair — attempt unauthorized read and write operations against the bucket using an S3-compatible client.
4. Remediate by issuing a properly scoped replacement key, and re-test the same operations to measure the change in exposure.
5. Translate the findings into concrete, general-purpose recommendations for cloud storage credential hygiene.

1.3 Scope and Constraints

All testing in this report was carried out exclusively against infrastructure owned and created by the author for this assignment: a single, purpose-built, private Backblaze B2 bucket (april-misconfig-lab-2026) and two Application Keys issued against it. No third-party systems, no production data, and no real personal information were involved at any stage. All file contents used in testing were synthetic placeholder data created for this lab. Live secret values captured in screenshots have been redacted in this report and the keys themselves are being rotated after submission, consistent with the recommendations in Section 8.

2. Literature and Tooling Review

This assessment draws on established industry frameworks for classifying and reasoning about the vulnerability class under test, rather than inventing new terminology for it.

2.1 MITRE ATT&CK

MITRE ATT&CK provides two directly relevant techniques used to frame this assessment. Technique T1552 (Unsecured Credentials) describes adversaries searching accessible locations — including public code repositories — for credentials such as API keys and access tokens that were not properly protected [1]. Technique T1530 (Data from Cloud Storage) describes adversaries accessing data from improperly secured cloud storage, typically because of misconfigured permissions or overly permissive credentials [2]. This project effectively reconstructs the T1552 → T1530 attack chain from the ground up: T1552 corresponds to the GitHub-leak stage of this lab, and T1530 corresponds to the subsequent unauthorized read/write stage against the bucket.

2.2 CIS Benchmarks and Cloud Hardening Guidance

The Center for Internet Security publishes benchmarks that, among other things, recommend least-privilege access to storage resources and periodic review of issued credentials against actual need [3]. While CIS does not publish a Backblaze-specific benchmark, the same least-privilege principle it applies to AWS S3 IAM policies is directly transferable to B2 Application Keys, since both models let an administrator scope a credential to a single bucket and a minimum permission set.

2.3 OWASP Cloud-Native Application Security

The OWASP Cloud-Native Application Security project lists insecure secrets management and misconfigured cloud storage among its top concerns for cloud-native systems, explicitly calling out hardcoded and overly broad credentials as a recurring finding in real assessments [4]. This project's over-permissive Application Key is a direct instance of that finding class.

2.4 CWE-798 — Use of Hard-Coded Credentials

The Common Weakness Enumeration catalogs CWE-798 for exactly the pattern simulated here: credentials embedded in a file that is then exposed, giving any party who obtains the file the same access as the legitimate application [5]. The .env file used in this assessment is a textbook instance of this weakness.

2.5 Secret-Leak Detection Tooling

GitHub's built-in secret scanning automatically checks pushed commits against known credential patterns from a large number of cloud providers and alerts repository owners when a likely secret is detected [6]. Open-source tools such as TruffleHog perform a similar function against arbitrary git history, including commits that predate a repository being made public, and are commonly run in CI pipelines or by security teams auditing an organization's repositories [9]. Neither tool was configured to intervene during this lab, by design, so that the full unauthorized-access chain could be observed uninterrupted; Section 8 recommends both as standing controls for a real environment.

2.6 Backblaze B2 and the S3-Compatible API

Backblaze B2 exposes an S3-compatible API alongside its native API, meaning standard S3 tooling — including the AWS Command Line Interface used throughout this assessment — can authenticate and operate against a B2 bucket by pointing at Backblaze's own regional endpoint rather than an AWS endpoint [7], [8]. This made it possible to test B2 exactly as an attacker would test any S3-compatible target, using freely available, industry-standard tooling rather than a Backblaze-specific script.

2.7 NIST SP 800-53

NIST SP 800-53 Revision 5 defines the AC-6 (Least Privilege) control family, which requires that information systems be configured to enforce the most restrictive set of rights needed to accomplish a task [10]. The remediation stage of this assessment (Section 6) is a direct, practical application of AC-6 to a cloud storage credential.

3. Methodology

3.1 Lab Setup

The lab consisted of four logical components: (1) a Windows 11 analyst workstation used to administer Backblaze B2 and run Git and the AWS CLI, (2) a private Backblaze B2 bucket acting as the target system, (3) a public GitHub repository used purely as the leak vector, and (4) an attacker-equivalent command-line session on the same workstation, kept logically separate through the use of distinct AWS CLI named profiles (leaked-key and scoped-key) so that each credential's capabilities could be tested in isolation. Figure 1 illustrates this topology and the order of operations.

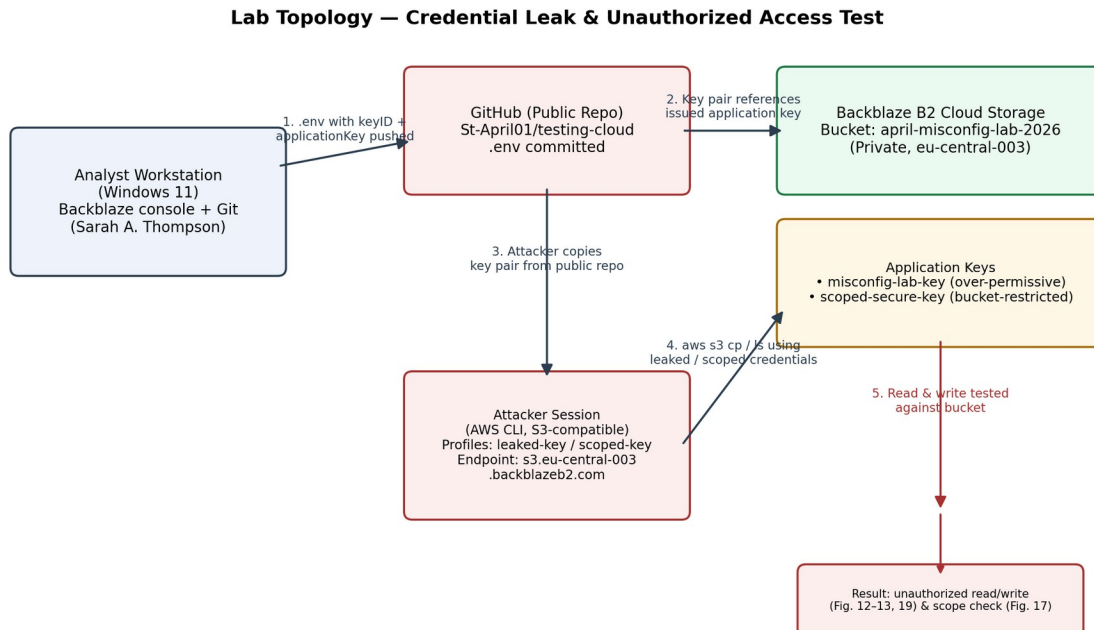


Figure 1. Lab topology showing the flow from credential creation, through the simulated GitHub leak, to unauthorized access testing and the subsequent remediation check.

3.2 Tools Used

- Backblaze B2 Cloud Storage — target platform, S3-compatible object storage.
- Git (v2.x) and GitHub — used to simulate accidental credential exposure via a public repository.
- AWS Command Line Interface v2.36.14 — used as a generic S3-compatible client against Backblaze's endpoint, not against any AWS account.
- Windows 11 Command Prompt / PowerShell — command execution environment.
- Brave browser — used to administer the Backblaze B2 console and GitHub.

3.3 Test Steps

1. Create an isolated, private Backblaze B2 bucket for the assessment.
2. Generate an Application Key with intentionally broad (account-wide, read + write) scope, representing a common real-world over-provisioning mistake.
3. Place the key pair in a .env file alongside other realistic application configuration values.
4. Commit and push the .env file to a new public GitHub repository, simulating an accidental leak.
5. Using only the leaked key pair, configure an AWS CLI profile and attempt unauthorized read (download) and write (upload) operations against the bucket via Backblaze's S3-compatible endpoint.
6. Generate a second Application Key, scoped to the single bucket rather than the whole account, as the remediation step.

7. Repeat the same read/write/list operations with the scoped key and compare the results against Step 5.
8. Analyze the differences and derive recommendations.

4. Implementation

4.1 Bucket Provisioning

A new bucket was created directly from the B2 Cloud Storage console (Figures 2–4). The bucket was left Private with default encryption and Object Lock disabled, since the objective of this assessment was to test credential-level exposure rather than bucket-policy exposure.

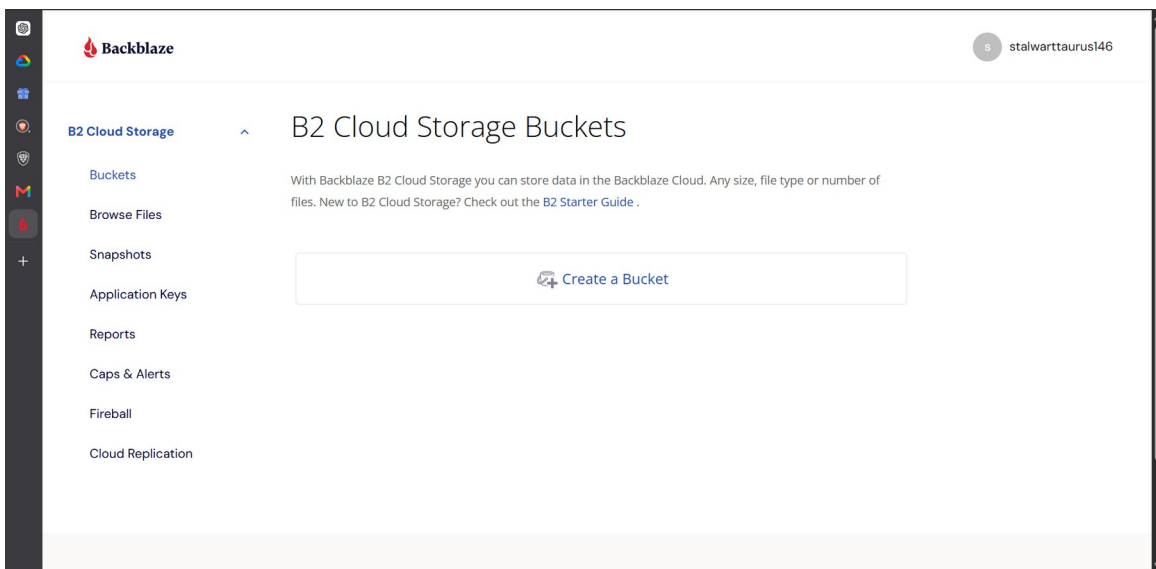


Figure 2. The B2 Cloud Storage Buckets dashboard prior to bucket creation.

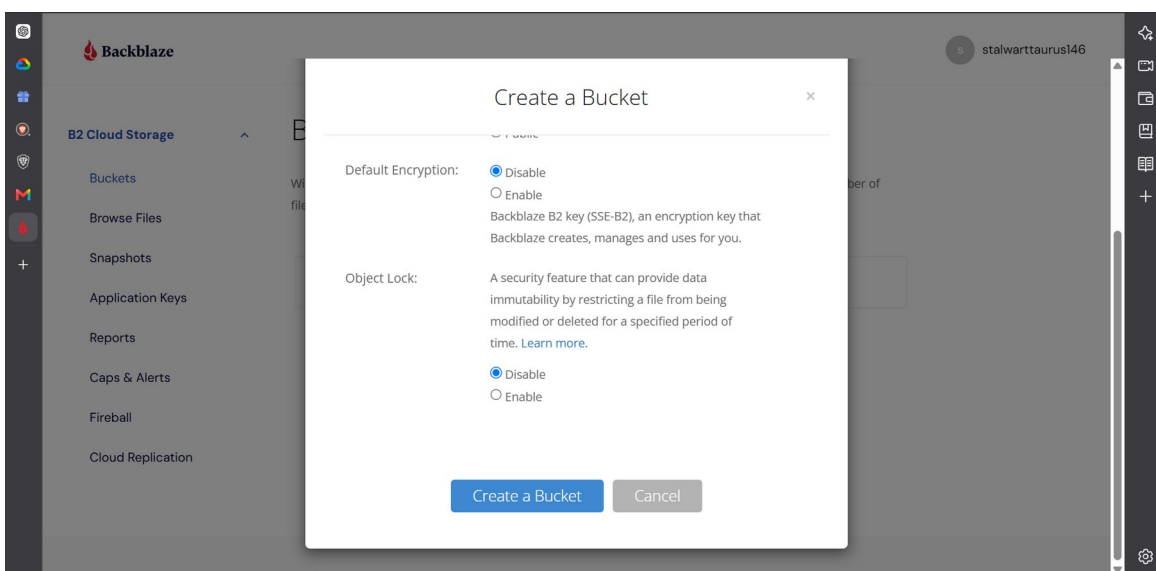


Figure 3. Bucket creation dialog; default encryption and Object Lock left disabled for this test bucket.

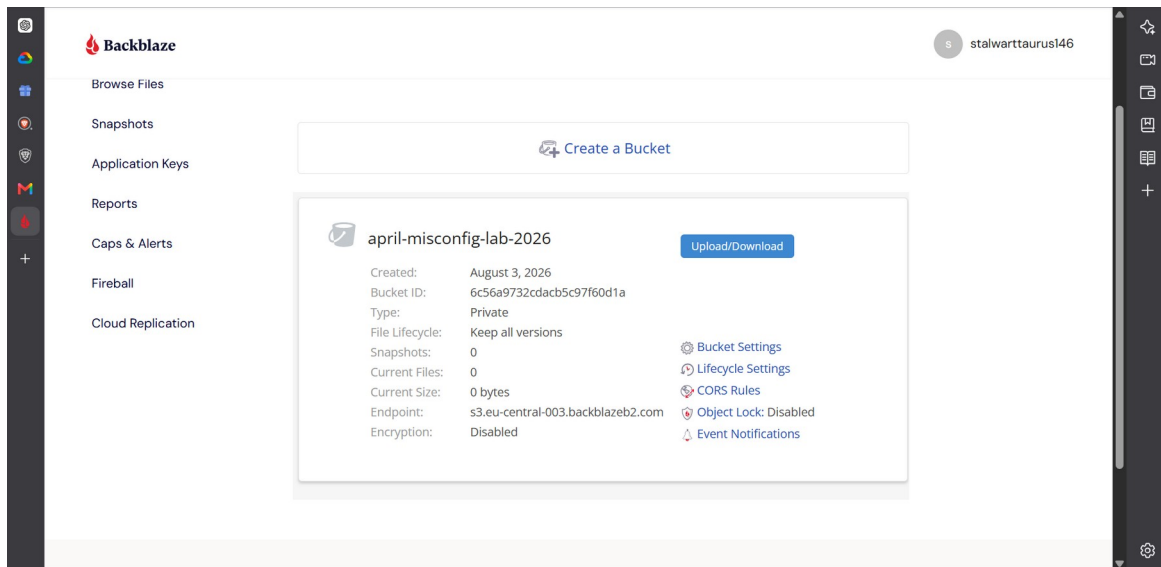


Figure 4. Confirmed bucket configuration: *april-misconfig-lab-2026*, Private, region *eu-central-003*, endpoint *s3.eu-central-003.backblazeb2.com*.

Bucket name	april-misconfig-lab-2026
Bucket ID	6c56a9732cdacb5c97f60d1a
Access type	Private
Region / Endpoint	eu-central-003 / s3.eu-central-003.backblazeb2.com
Encryption	Disabled

4.2 Intentional Misconfiguration — Over-Permissive Application Key

An Application Key named *misconfig-lab-key* was generated with Read and Write access and account-wide scope rather than being restricted to the single test bucket. This mirrors a common real-world shortcut taken during development, where a broadly scoped key is issued for convenience and never revisited.

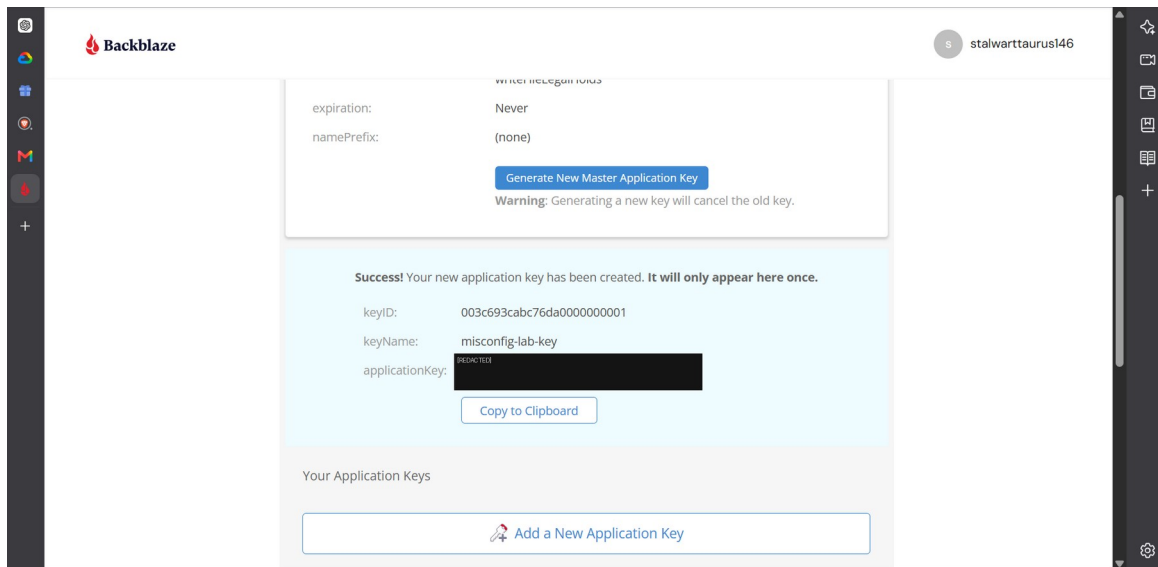


Figure 5. Application Key "misconfig-lab-key" generated (keyID 003c693cab76da0000000001). The applicationKey secret has been redacted in this report; the live value was captured only in the original lab session.

4.3 Simulating Credential Leakage via a Public Repository

To simulate the most common real-world leak vector for this class of credential, the keyID and applicationKey were placed inside a .env configuration file alongside other realistic (but synthetic) application settings, then committed to a local Git repository and pushed to a new public GitHub repository.

```
APP_NAME=inventory-sync-service
B2_KEY_ID=003c693cab76da0000000001
B2_APPLICATION_KEY=<redacted in this report>
B2_BUCKET_NAME=april-misconfig-lab-2026
B2_ENDPOINT=https://s3.eu-central-003.backblazeb2.com
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Sarah April Thompson\Videos\cloud> git init
Initialized empty Git repository in C:/Users/Sarah April Thompson/Videos/cloud/.git/
PS C:\Users\Sarah April Thompson\Videos\cloud> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Sarah April Thompson\Videos\cloud> |

```

Figure 6. Local Git repository initialized; .env shown as an untracked file prior to commit.

```

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Sarah April Thompson\Videos\cloud> git add .
warning: in the working copy of '.env', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Sarah April Thompson\Videos\cloud> git commit -m "added .env"
[master (root-commit) be7813c] added .env
 1 file changed, 25 insertions(+)
 create mode 100644 .env
PS C:\Users\Sarah April Thompson\Videos\cloud> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Sarah April Thompson\Videos\cloud> git remote add origin https://github.com/St-April01/testing-cloud.git
PS C:\Users\Sarah April Thompson\Videos\cloud> git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 629 bytes | 629.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/St-April01/testing-cloud.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
PS C:\Users\Sarah April Thompson\Videos\cloud> |

```

Figure 7. .env committed ("added .env") and pushed to the remote origin on GitHub.

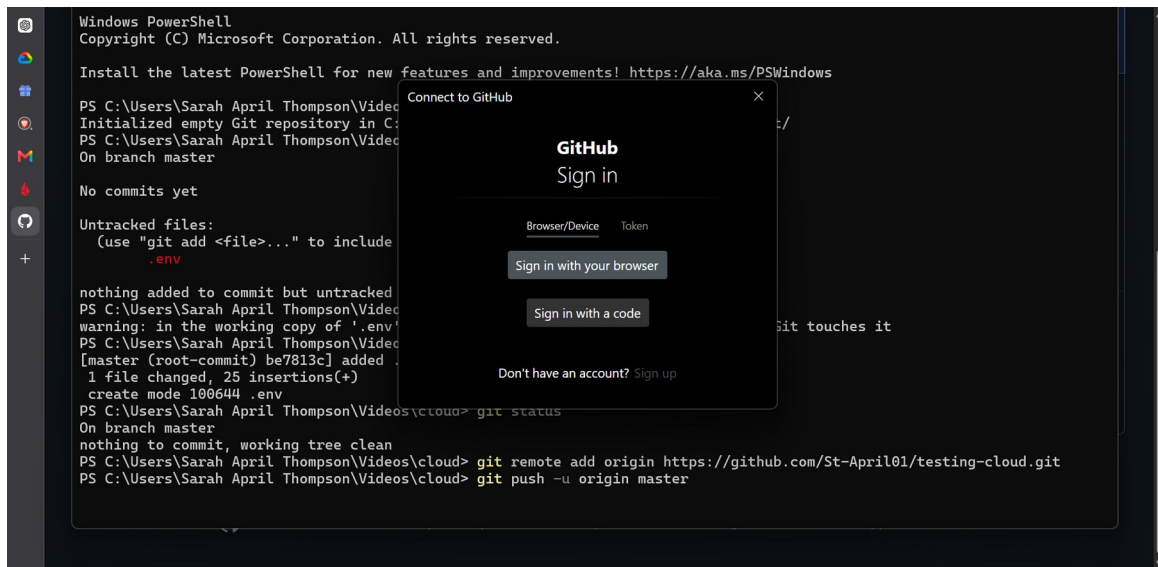


Figure 8. GitHub device authentication prompt encountered during git push.

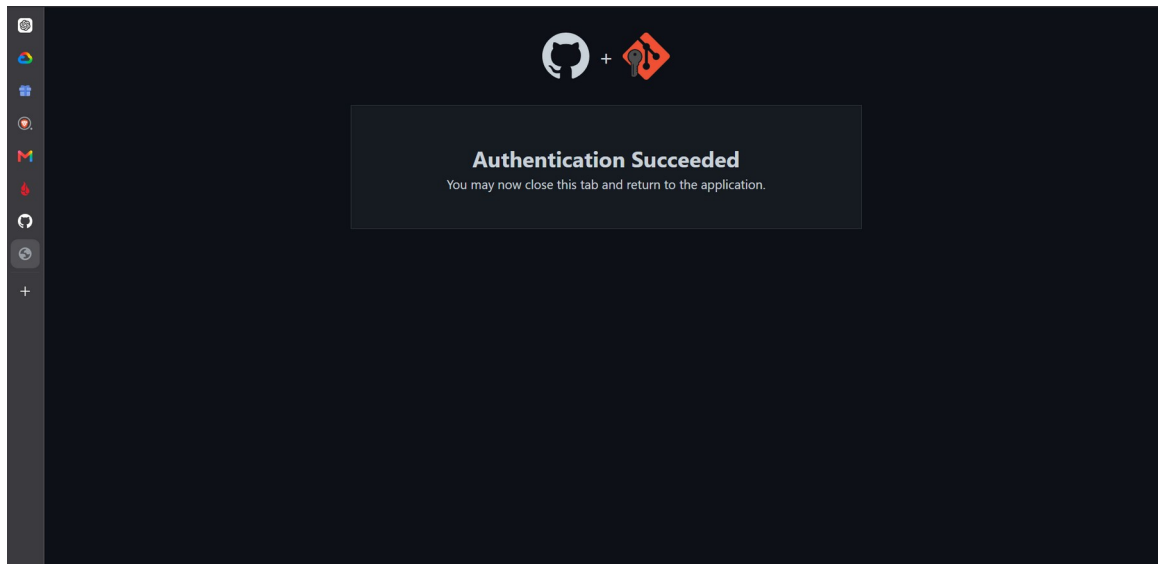


Figure 9. GitHub authentication succeeded, allowing the push to complete.

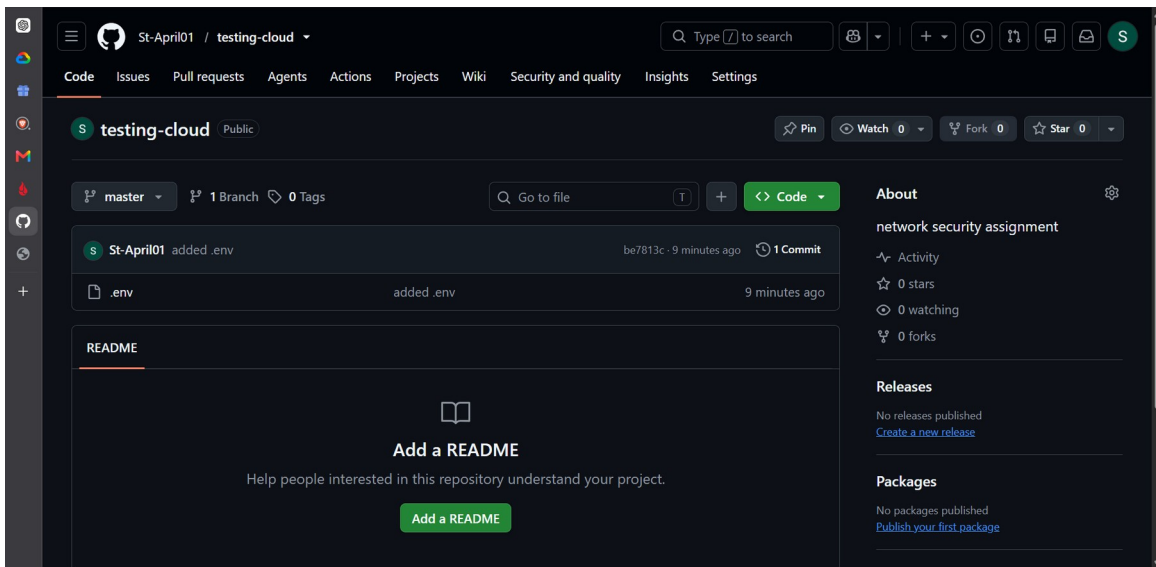


Figure 10. The `.env` file now visible in the public GitHub repository `St-April01/testing-cloud`, simulating discovery by an outside party. A real attacker could locate this using GitHub's code search or automated repository-scanning bots.

5. Results and Findings — Unauthorized Access

With only the leaked `keyID` and `applicationKey` in hand, an attacker-equivalent AWS CLI profile named `leaked-key` was configured. No other information about the account was assumed or used.

```

Command Prompt
Microsoft Windows [Version 10.0.26100.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sarah April Thompson>jj
'jj' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Sarah April Thompson>aws --version
aws-cli/2.36.14 Python/3.14.6 Windows/x64 AMD64

C:\Users\Sarah April Thompson> aws configure --profile leaked-key

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws log
in'.

AWS Access Key ID [None]: 003c693cab76da0000000001
AWS Secret Access Key [None]: K[REDACTED]
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>

```

Figure 11. `aws configure --profile leaked-key` using the leaked credential pair. The secret value is redacted in this report.

5.1 Confirming the Target File

The Backblaze console was used only to confirm the identity of a file already present in the private bucket, for later comparison against what the leaked key could retrieve; this step represents information available to the legitimate owner, not to the attacker.

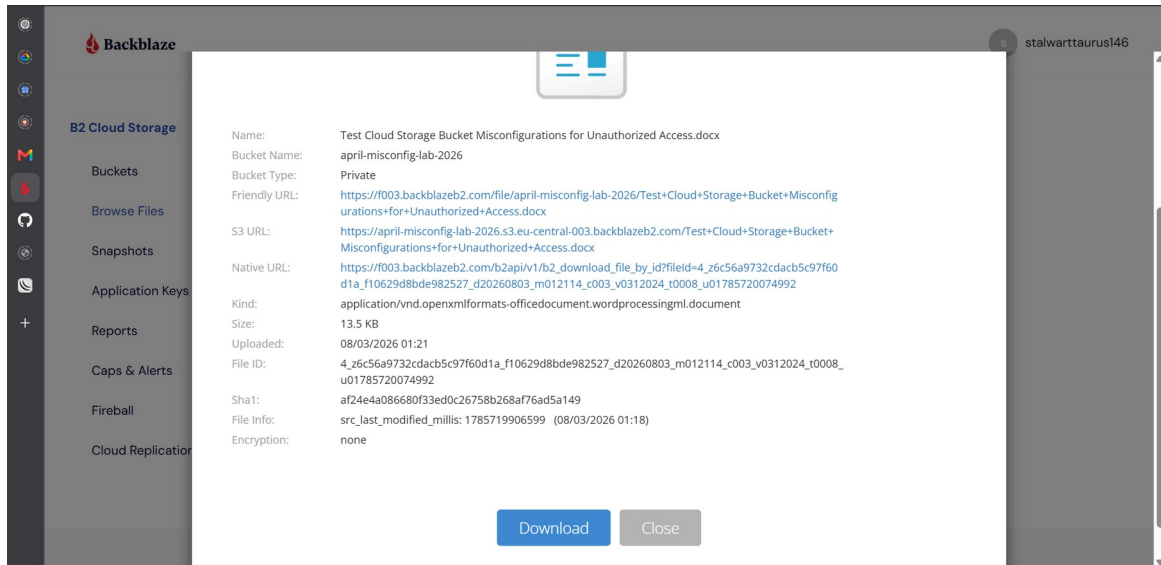


Figure 12. File details for the target document stored in the private bucket, used to confirm the retrieved copy matched exactly.

5.2 Unauthorized Read Access

A download request was issued against the private bucket using only the leaked key. Two initial attempts against incorrect regional endpoints failed with `InvalidAccessKeyId` or connection errors — an artifact of endpoint/region mismatch during testing, not a security control — but the correctly targeted request succeeded.

```
aws s3 cp "s3://april-misconfig-lab-2026/<file>.docx" "./stolen_file.docx" \
  --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
```

```

Microsoft Windows [Version 10.0.26100.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sarah April Thompson>aws --version
aws-cli/2.36.14 Python/3.14.6 Windows/11 exe/AMD64

C:\Users\Sarah April Thompson>aws s3 cp "s3://april-misconfig-lab-2026/Test Cloud Storage Bucket Misconfigurations for Unauthorized Access.docx" ".\stolen_file.docx" --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
download: s3://april-misconfig-lab-2026/Test Cloud Storage Bucket Misconfigurations for Unauthorized Access.docx to .\stolen_file.docx

C:\Users\Sarah April Thompson>echo "This file was injected by an unauthorized key" > injected.txt

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000001' is not valid

C:\Users\Sarah April Thompson>

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000001' is not valid

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-003.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/april-misconfig-lab-2026/injected.txt"

C:\Users\Sarah April Thompson>

```

Figure 13. The unauthorized download succeeds ("download: ... to .\stolen_file.docx"). The two preceding upload attempts against us-west-004 and us-west-003 endpoints fail — expected, since the bucket is actually hosted in eu-central-003.

5.3 Unauthorized Write Access

To test whether the leaked key also permitted tampering with stored data, a file (injected.txt) was created locally and uploaded to the bucket using the same leaked key and the correct regional endpoint.

```

aws s3 cp injected.txt s3://april-misconfig-lab-2026/ \
--endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key

```

```

Microsoft Windows [Version 10.0.26100.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sarah April Thompson>aws --version
aws-cli/2.36.14 Python/3.14.6 Windows/11 exe/AMD64

C:\Users\Sarah April Thompson>aws s3 cp "s3://april-misconfig-lab-2026/Test Cloud Storage Bucket Misconfigurations for Unauthorized Access.docx" ".\stolen_file.docx" --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
download: s3://april-misconfig-lab-2026/Test Cloud Storage Bucket Misconfigurations for Unauthorized Access.docx to .\stolen_file.docx

C:\Users\Sarah April Thompson>echo "This file was injected by an unauthorized key" > injected.txt

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000001' is not valid

C:\Users\Sarah April Thompson>

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000001' is not valid

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-003.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/april-misconfig-lab-2026/injected.txt"

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
upload: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt

C:\Users\Sarah April Thompson>

```

Figure 14. Once the correct endpoint (eu-central-003) is used, the upload succeeds ("upload: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt") — confirming the leaked key grants unauthorized write access, not just read.

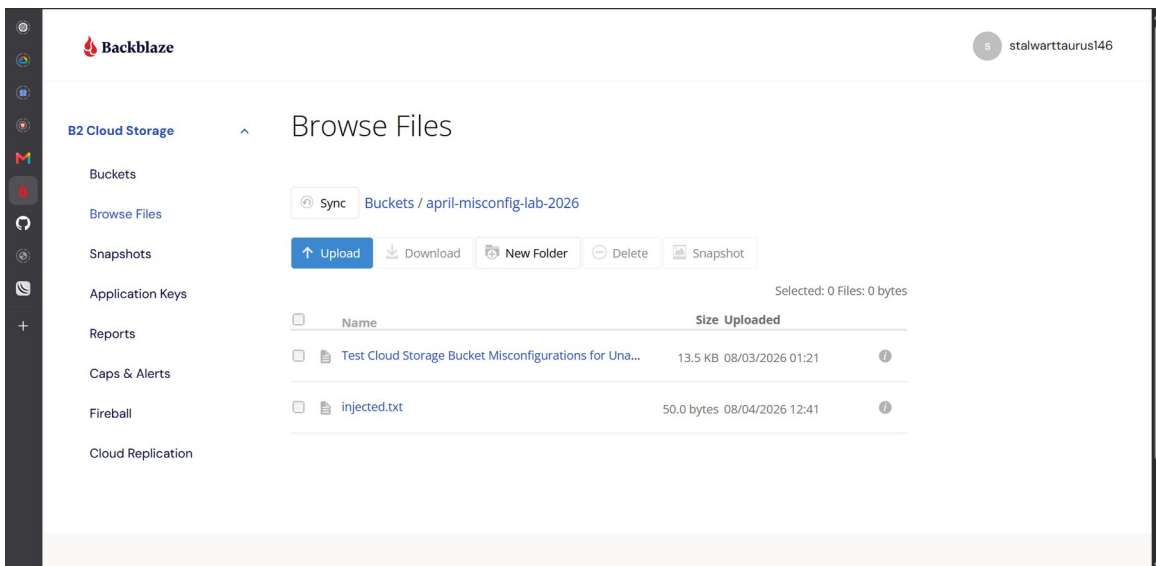


Figure 15. Bucket contents in the Backblaze console confirming injected.txt was written by the leaked-key session, alongside the original file.

These two results together confirm that the over-permissive key, once leaked, gave an unauthorized party full read and write access to the bucket's contents — equivalent to the access level of the legitimate application, despite never having been issued to any legitimate party.

6. Remediation and Validation

To remediate, a second Application Key, `scoped-secure-key`, was generated and restricted to the single bucket rather than the whole account. The intent was to test whether bucket-level scoping alone — independent of the Read/Write permission level chosen — measurably reduces what a leaked credential could do.

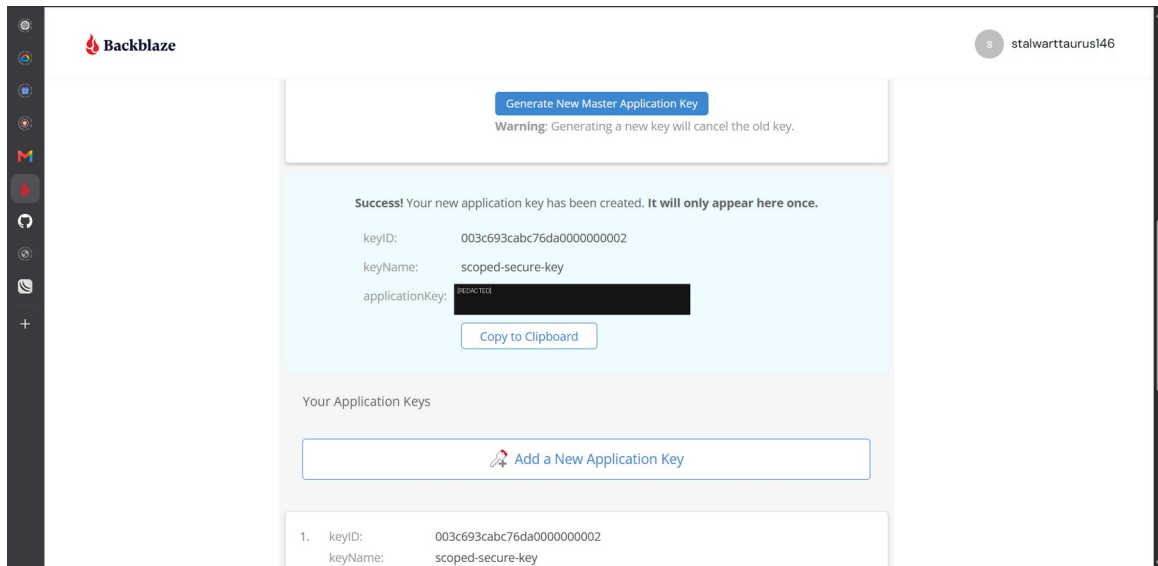


Figure 16. New Application Key "scoped-secure-key" generated (keyID 003c693cab76da0000000002), restricted to the single test bucket. The applicationKey secret is redacted in this report.

6.1 Testing the Scoped Key

The same categories of operation tested against the leaked key were repeated against the scoped key, using a separate AWS CLI profile (`scoped-key`) so the two credentials could never be confused during testing.

```

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.us-west-003.backblazeb2.com --profile leaked-key
upload failed: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/april-misconfig-lab-2026/injected.txt"

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
upload: .\injected.txt to s3://april-misconfig-lab-2026/injected.txt

C:\Users\Sarah April Thompson>aws configure --profile scoped-key

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: 003c693cab76da0000000002
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws configure --profile scoped-key
AWS Access Key ID [*****0002]:
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-003.backblazeb2.com --profile scoped-key

aws: [ERROR]: Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/"

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key

aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da0000000002' is not valid

C:\Users\Sarah April Thompson>

```

Figure 17. `aws configure --profile scoped-key`, followed by `ListBuckets` attempts against incorrect endpoints (connection error / `InvalidAccessKeyId`, consistent with the endpoint-region mismatch seen earlier). Secret value redacted.

```

C:\Users\Sarah April Thompson>aws configure --profile scoped-key

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: 003c693cab76da0000000002
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws configure --profile scoped-key
AWS Access Key ID [*****0002]:
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-003.backblazeb2.com --profile scoped-key

aws: [ERROR]: Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/"

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key

aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da0000000002' is not valid

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key

aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets operation: not entitled

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://yourname-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key
upload failed: .\injected.txt to s3://yourname-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000002' is not valid

C:\Users\Sarah April Thompson>

```

Figure 18. Against the correct endpoint (eu-central-003), `ListBuckets` is refused with `AccessDenied`: "not entitled" — the scoped key cannot enumerate the account's other buckets. A subsequent upload attempt using an incorrect placeholder bucket name also fails, as expected. Secret value redacted.

```

AWS Access Key ID [None]: 003c693cab76da0000000002
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws configure --profile scoped-key
AWS Access Key ID [*****0002]:
AWS Secret Access Key [None]:
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-003.backblazeb2.com --profile scoped-key
aws: [ERROR]: Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/"

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key
aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da0000000002' is not valid

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets operation: not entitled

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://yourname-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key
upload failed: .\injected.txt to s3://yourname-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da0000000002' is not valid

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da0000000001' is not valid

C:\Users\Sarah April Thompson>

```

Figure 19. For comparison, the original leaked-key profile is also retried against an incorrect endpoint, failing with *InvalidAccessKeyId* — reflecting the endpoint mismatch during this troubleshooting step, not key revocation. Secret value redacted.

6.2 Confirming Read and Write Behaviour Within the Scoped Bucket

A final pair of tests, run against the correct endpoint and the correct bucket name, confirmed the actual read/write capability of the scoped key within its own bucket.

```

aws s3 cp s3://april-misconfig-lab-2026/injected.txt ./test.txt \
--endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key
aws s3 cp injected.txt s3://april-misconfig-lab-2026/ \
--endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key

```

```

AWS Secret Access Key [None]: REDACTED
Default region name [None]:
Default output format [None]:

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-003.backblazeb2.com --profile scoped-key
aws: [ERROR]: Could not connect to the endpoint URL: "https://s3.us-west-003.backblazeb2.com/"

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key
aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da000000000002' is not valid

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets operation: not entitled

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://yourname-misconfig-lab-2026/ --endpoint-url https://s3.us-west-004.backblazeb2.com --profile scoped-key
upload failed: .\injected.txt to s3://yourname-misconfig-lab-2026/injected.txt An error occurred (InvalidAccessKeyId) when calling the PutObject operation: The key '003c693cab76da000000000002' is not valid

C:\Users\Sarah April Thompson>aws s3 ls --endpoint-url https://s3.us-west-004.backblazeb2.com --profile leaked-key
aws: [ERROR]: An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The key '003c693cab76da000000000001' is not valid

C:\Users\Sarah April Thompson>aws s3 cp s3://april-misconfig-lab-2026/injected.txt .\test.txt --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key
download: s3://april-misconfig-lab-2026/injected.txt to .\test.txt

C:\Users\Sarah April Thompson>aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile scoped-key
Completed 50 Bytes/50 Bytes (13 Bytes/s) with 1 file(s) remaining
C:\Users\Sarah April Thompson>

```

Figure 20. Within its own bucket and against the correct endpoint, the scoped key both downloads ("download: ... to .\test.txt") and uploads ("Completed 50 Bytes/50 Bytes") successfully — while the earlier figure confirmed it cannot list other buckets on the account. Secret value redacted.

This is the key remediation evidence of the assessment: bucket-level scoping was successfully verified (the scoped key cannot enumerate or reach any bucket other than the one it was issued for), but the permission level within that bucket was not reduced — the scoped key retained full read and write access, because Read+Write was the permission level selected when the key was created. Scope and permission level are configured independently in Backblaze B2, and this test shows that tightening one does not automatically tighten the other.

7. Analysis and Recommendations

7.1 What the Results Mean

The assessment confirms a complete, realistic attack chain: a credential over-provisioned at creation time, leaked through an entirely ordinary developer mistake (committing a .env file to a public repository), and then used exactly as the legitimate application would have used it, with no additional exploitation required. Nothing about Backblaze B2 itself was exploited — every operation performed was a standard, documented API call, authenticated with a key that was never meant to leave the developer's machine. This is consistent with the pattern MITRE ATT&CK describes across T1552 and T1530 [1], [2], and with OWASP's and CWE's characterization of hardcoded, over-scoped credentials as a top cloud-native risk [4], [5].

The remediation stage shows that scoping a key to a single bucket is a real and independently verifiable control — the AccessDenied: "not entitled" response is concrete evidence that the blast radius of a future leak involving this key would now be limited to one bucket rather than the entire account. However, the same test also shows that scoping alone is not sufficient:

because the permission level was left at Read+Write, a leak of the new key would still allow an attacker to read and tamper with everything in that one bucket. Full remediation, per the principle of least privilege (NIST SP 800-53 AC-6 [10]), requires setting both controls deliberately based on actual application need, rather than assuming one implies the other.

7.2 Recommendations

- Scope every Application Key to the single bucket it is meant to serve, never to the whole account, as verified in Section 6.1.
- In addition to bucket scoping, select the minimum permission level the application actually needs (Read-only where write is not required); this test used Read+Write throughout and did not verify a Read-only configuration, which should be validated separately before being relied upon in production.
- Never commit credentials to source control, including private repositories; use a secrets manager or CI/CD-injected environment variables instead of .env files that could be committed by mistake.
- Enable GitHub secret scanning [6] and/or run TruffleHog [9] or an equivalent tool against repositories on a recurring basis to catch accidental credential commits before they are exploited.
- Rotate and revoke any key immediately once a leak is suspected or confirmed; the misconfig-lab-key generated for this assessment is being deleted from the Backblaze account after submission.
- Periodically audit issued Application Keys against actual application need to catch scope or permission creep over time, consistent with CIS least-privilege guidance [3].
- Enable bucket access logging so that unauthorized access attempts, such as those performed in this assessment, would be visible to the account owner going forward.

7.3 Limitations

This assessment tested a single cloud storage provider (Backblaze B2) and a single credential type (Application Keys). While the underlying weakness class (CWE-798, over-provisioned credentials) generalizes broadly across S3-compatible providers, the exact console workflow and error messages shown here are Backblaze-specific and may differ elsewhere. The assessment also did not test a Read-only scoped key, secret-scanning tool integration, or bucket access logging in practice; these are identified as recommendations rather than validated controls, and are natural next steps for further work.

8. Conclusion

This project demonstrated, under controlled lab conditions, the complete lifecycle of a credential-driven cloud storage breach: an over-permissive Backblaze B2 Application Key was created, leaked through a realistic and common mistake (a .env file committed to a public GitHub repository), and then used by an unauthorized party to both read and write data inside

a private bucket, entirely through standard, documented API operations. A remediation key scoped to a single bucket was then generated and validated, confirming that bucket-level scoping is an effective and independently testable control — the scoped key could not enumerate other buckets on the account — while also surfacing an important nuance: scope and permission level are separate settings, and restricting one does not restrict the other. The exercise reinforces that credential hygiene — least-privilege scoping, minimum-necessary permissions, secret-leak prevention, and prompt rotation — is not a theoretical best practice but a directly testable, directly observable control whose absence produces exactly the outcome demonstrated in Section 5 of this report.

References

- [1] MITRE ATT&CK, "T1552 — Unsecured Credentials," MITRE Corporation. [Online]. Available: <https://attack.mitre.org/techniques/T1552/>
- [2] MITRE ATT&CK, "T1530 — Data from Cloud Storage," MITRE Corporation. [Online]. Available: <https://attack.mitre.org/techniques/T1530/>
- [3] Center for Internet Security, "CIS Benchmarks," CIS. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks>
- [4] OWASP Foundation, "OWASP Cloud-Native Application Security Top 10." [Online]. Available: <https://owasp.org/www-project-cloud-native-application-security-top-10/>
- [5] MITRE, "CWE-798: Use of Hard-coded Credentials," Common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/798.html>
- [6] GitHub, "About secret scanning," GitHub Docs. [Online]. Available: <https://docs.github.com/en/code-security/secret-scanning>
- [7] Backblaze Inc., "B2 Cloud Storage S3-Compatible API," Backblaze Documentation. [Online]. Available: <https://www.backblaze.com/b2/docs/>
- [8] Amazon Web Services, "AWS Command Line Interface Reference," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/cli/>
- [9] Truffle Security, "TruffleHog: Find leaked credentials," GitHub repository. [Online]. Available: <https://github.com/trufflesecurity/trufflehog>
- [10] National Institute of Standards and Technology, Security and Privacy Controls for Information Systems and Organizations, NIST Special Publication 800-53, Revision 5, Sep. 2020.

Appendix A — Full Command Log (Leaked Key)

```
aws --version
aws-cli/2.36.14 Python/3.14.6 Windows/11 exe/AMD64
aws configure --profile leaked-key
  AWS Access Key ID [None]: 003c693cab76da000000000001
  AWS Secret Access Key [None]: <redacted>
aws s3 cp "s3://april-misconfig-lab-2026/<file>.docx" "./stolen_file.docx" --
endpoint-url https://s3.eu-central-003.backblazeb2.com --profile leaked-key
  -> download succeeded
echo "This file was injected by an unauthorized key" > injected.txt
aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url
https://s3.eu-central-003.backblazeb2.com --profile leaked-key
  -> upload succeeded
```

Appendix B — Full Command Log (Scoped Key)

```
aws configure --profile scoped-key
  AWS Access Key ID [None]: 003c693cab76da000000000002
  AWS Secret Access Key [None]: <redacted>
aws s3 ls --endpoint-url https://s3.eu-central-003.backblazeb2.com --profile
scoped-key
  -> [ERROR]: AccessDenied: "not entitled"
aws s3 cp s3://april-misconfig-lab-2026/injected.txt ./test.txt --endpoint-url
https://s3.eu-central-003.backblazeb2.com --profile scoped-key
  -> download succeeded
aws s3 cp injected.txt s3://april-misconfig-lab-2026/ --endpoint-url
https://s3.eu-central-003.backblazeb2.com --profile scoped-key
  -> Completed 50 Bytes/50 Bytes (upload succeeded)
```

Appendix C — .env File Used to Simulate the Leak

```
APP_NAME=inventory-sync-service
APP_ENV=production
B2_KEY_ID=003c693cab76da000000000001
B2_APPLICATION_KEY=<redacted>
B2_BUCKET_NAME=april-misconfig-lab-2026
B2_ENDPOINT=https://s3.eu-central-003.backblazeb2.com
```